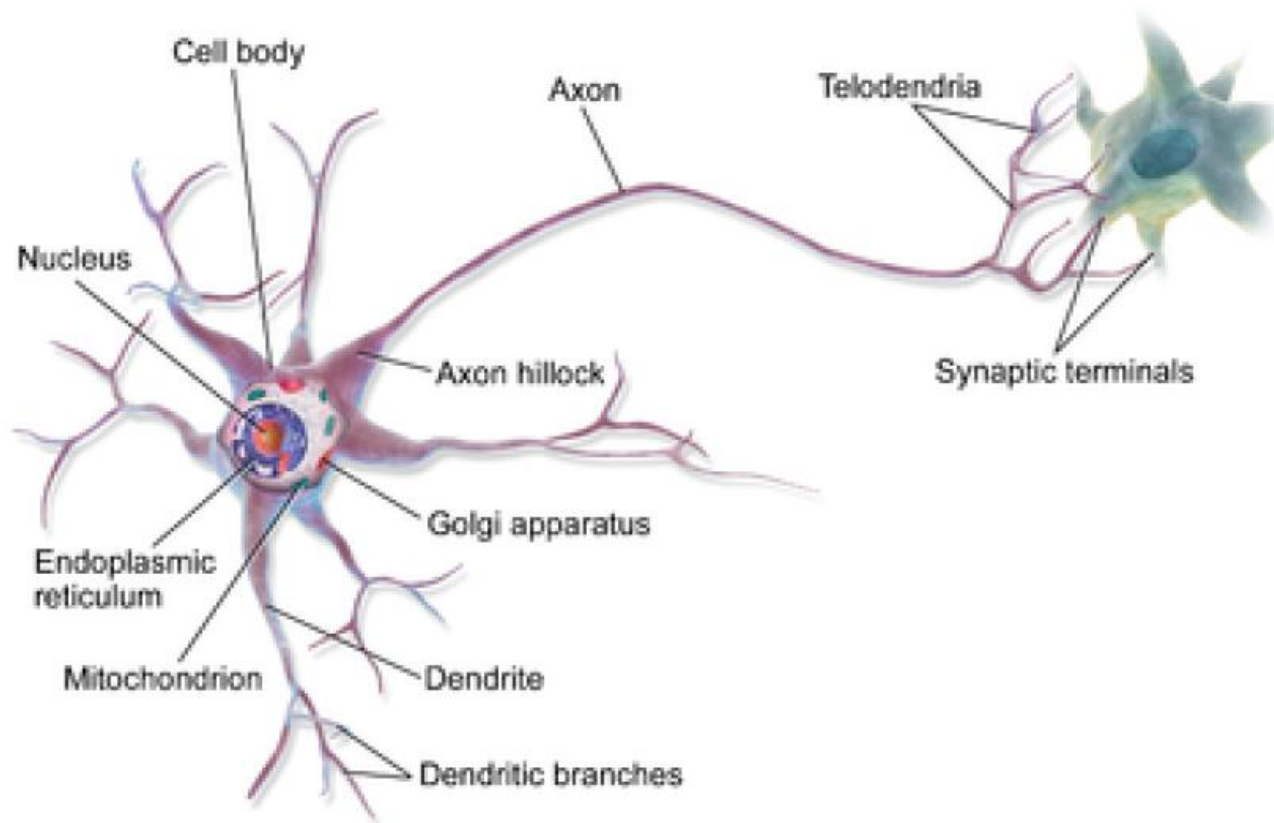


# Deep Learning

## **Lecture 2 - The Artificial Neuron (MP Neuron and Perceptron)**

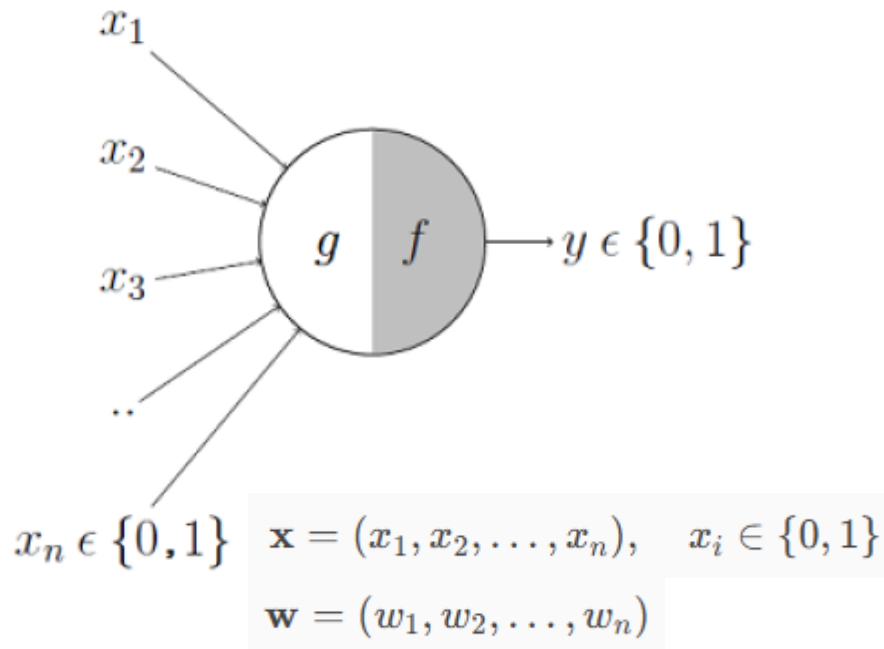
# The Neuron

- ~90 billion neurons in the human brain



# Threshold Logic Unit

- First mathematical neuron model
- McCulloch & Pitts (1943)
- Boolean inputs and output



$$z = \sum_{i=1}^n w_i x_i$$
$$y = \begin{cases} 1, & \text{if } z \geq \theta \\ 0, & \text{if } z < \theta \end{cases}$$

$\theta$  is the threshold

# Threshold Logic Unit

## Excitatory and Inhibitory Inputs

Inputs are divided into two disjoint sets:

**Excitatory inputs:** increase the chance of firing  $w_i > 0$

**Inhibitory inputs:** suppress neuron activation  $w_j < 0$

In M P Neurons, Weights are fixed and non-negative (often binary or integer values).

Inhibition is not represented using negative weights. Instead, it uses explicit inhibitory inputs

**hard logical rule :** If any inhibitory input is ON, the neuron does NOT fire

# Boolean Functions

- Counting Excitatory Signals

$$\mathbb{1}(\sum_i x_i \geq \theta)$$

- AND
- OR
- NOT

# Boolean Functions

- Counting Excitatory Signals

$$\mathbb{1}(\sum_i x_i \geq \theta)$$

- AND  $\theta = n$
- OR  $\theta = 1$
- NOT  $\theta = 0$

| Input $x$ | Inhibitory active? | Output |
|-----------|--------------------|--------|
| 0         | No                 | 1      |
| 1         | Yes                | 0      |

# Implementing Logic

- $xy'$
- NOR

# Implementing Logic

- $xy'$
- $x$ : excitatory input,  $y$ : inhibitory input ,  $\theta = 1$
- NOR



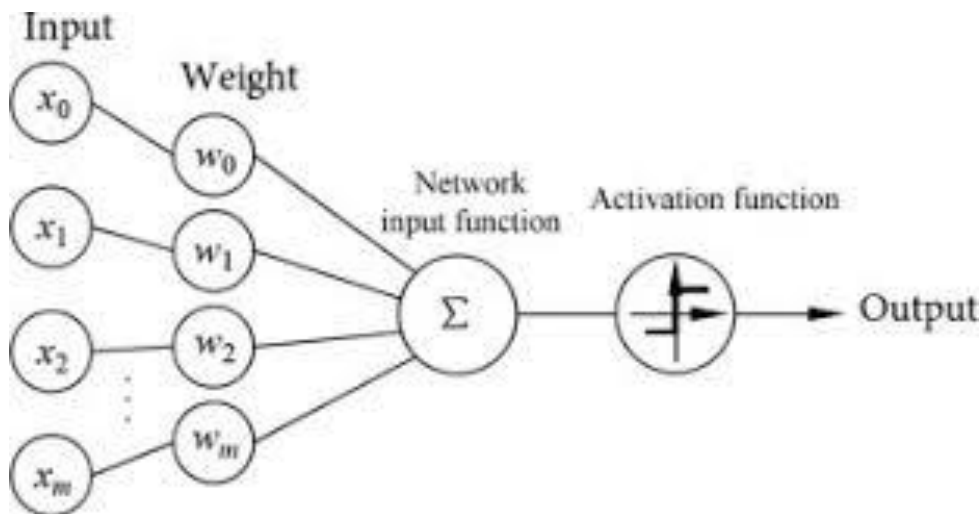
# What does one unit do?

- Learns linear separation
  - line in 2D, plane in 3D, hyperplane in higher dimensions
  - $Z = \theta$  defines a hyperplane (line in 2D, plane in 3D)
  - $Z \geq \theta$  classified as 1 and rest 0
  - No learning – heuristic setup

$$z = \sum_{i=1}^n w_i x_i$$

# Perceptron

- Frank Rosenblatt 1957 (American Psychologist)
- Very crude biological model- computation and learning
- Similar to MP neuron - Performs linear classification
- Inputs can be real, weights can be different for different i/p components



$$f(x) = \begin{cases} 1 & \text{when } \sum_i w_i x_i + b \geq 0 \\ 0 & \text{else} \end{cases}$$

# Perceptron

- For simplicity we consider +1 and -1 responses



$$\sigma(x) = \begin{cases} 1 & \text{when } x \geq 0 \\ -1 & \text{else} \end{cases}$$

$$f(\mathbf{x}) = \sigma(\mathbf{w}^T \cdot \mathbf{x} + b)$$

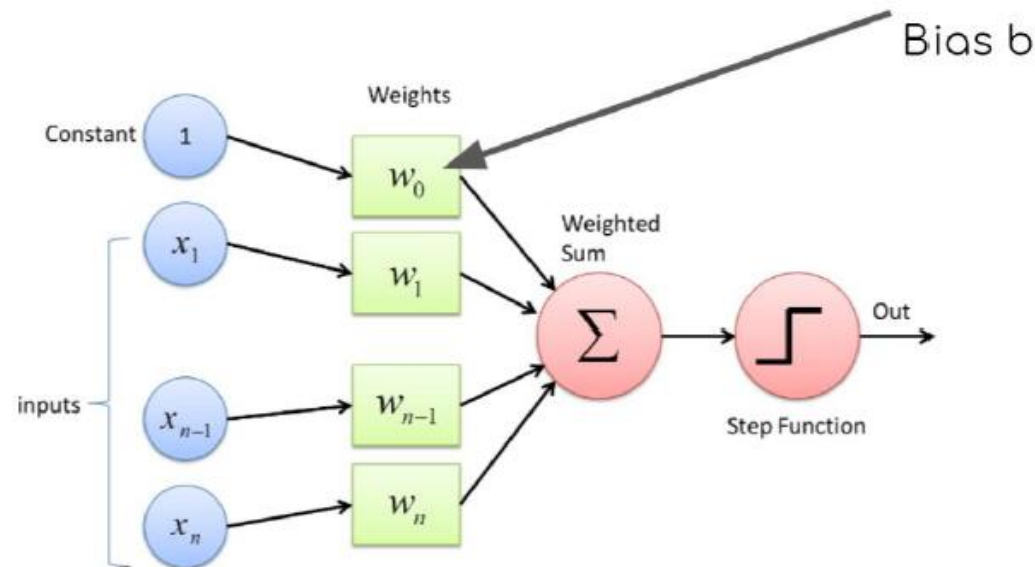
- In general,  $\sigma(\cdot)$  that follows a linear operation is called an activation function
- $\mathbf{w}$  are referred to as weights and  $b$  as the bias

# Perceptron vs MP Neuron

- Perceptron is more general computational model
- Inputs can be real
- Weights are different on the input components
- Mechanism for learning the weights

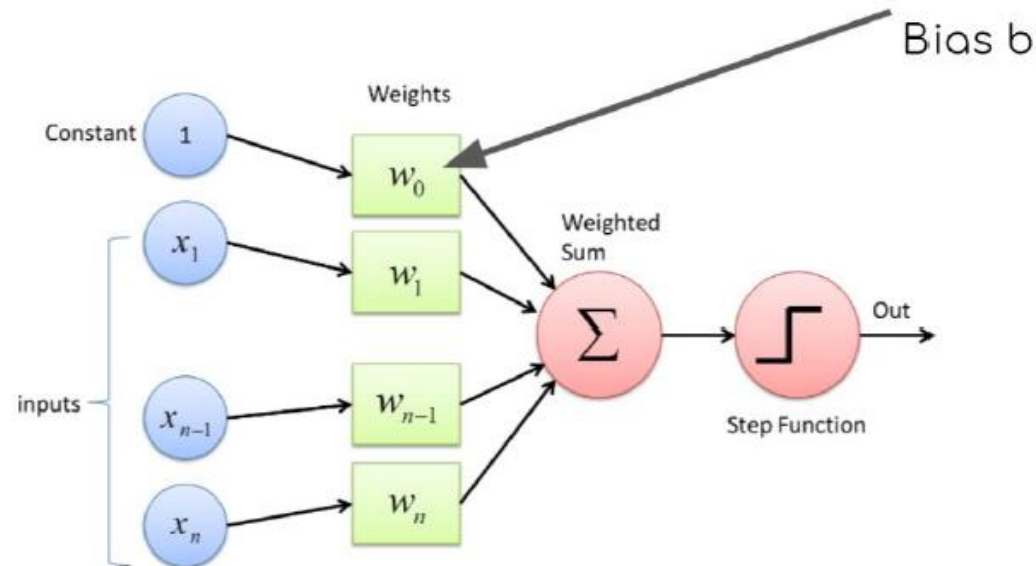
# Weights and Bias

- Why are weights important
- determine the importance of each input, prioritize certain features over others



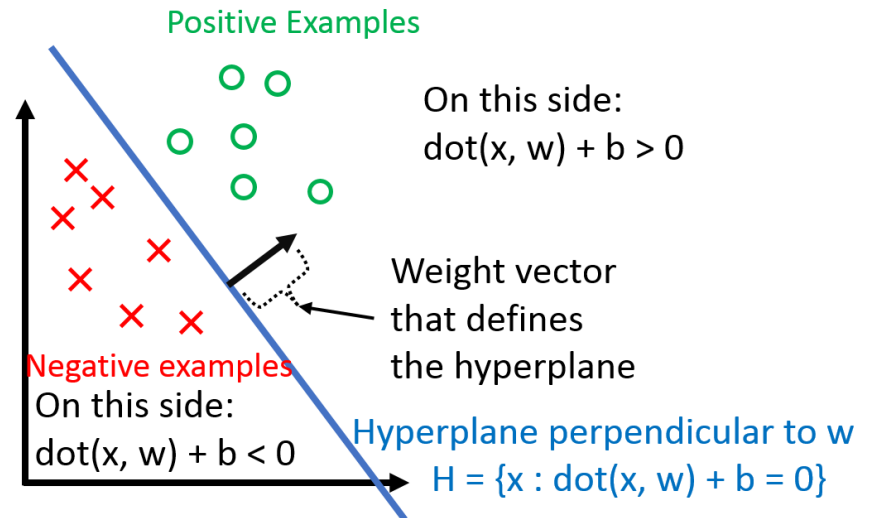
# Weights and Bias

- Why are bias important
- Bias ( $b$ ) serves the function of threshold
- It allows the model to shift the decision boundary



# Perceptron Geometry

- Linear decision boundary  $w \cdot x + b = 0$ , defines a linear surface that separates the input space.
- weight vector is perpendicular (normal) to the decision boundary. Changing the direction of  $w$  rotates the boundary. Bias shifts the decision boundary.
- Learning = adjusting weights and bias



# Learning Algorithm

Initialize weight vector  $w \leftarrow 0$

Initialize bias  $b \leftarrow 0$

for epoch = 1 to T do

  for each training example  $(x_i, y_i)$  in D do

    if  $y_i (w \cdot x_i + b) \leq 0$  then

**$w \leftarrow w + \eta y_i x_i$**

**$b \leftarrow b + \eta y_i$**

    end if

  end for

end for

return  $w, b$

Check for misclassification

If a positive example is misclassified  $\rightarrow$  move the boundary towards it

If a negative example is misclassified  $\rightarrow$  move the boundary away from it



# Perceptron learning rule

- Suppose we see a misclassified example  $(\mathbf{x}, y)$  where  $y(\mathbf{w}^\top \mathbf{x} + b) \leq 0$ .

# Perceptron learning rule

- Suppose we see a misclassified example  $(\mathbf{x}, y)$  where  $y(\mathbf{w}^\top \mathbf{x} + b) \leq 0$ .
- We want to update  $(\mathbf{w}, b)$  so that the signed margin increases:  
 $y(\mathbf{w}\mathbf{x}+b)$  should go up.

# Perceptron learning rule

- Suppose we see a misclassified example  $(\mathbf{x}, y)$  where  $y(\mathbf{w}^\top \mathbf{x} + b) \leq 0$ .
- We want to update  $(\mathbf{w}, b)$  so that the signed margin increases:  $y(\mathbf{w}\mathbf{x}+b)$  should go up.
- $\mathbf{w}' = \mathbf{w} + \Delta \mathbf{w}, b' = b + \Delta b$ .
- New signed margin

$$y(\mathbf{w}'^\top \mathbf{x} + b') = y((\mathbf{w} + \Delta \mathbf{w})^\top \mathbf{x} + (b + \Delta b)) = y(\mathbf{w}^\top \mathbf{x} + b) + y(\Delta \mathbf{w}^\top \mathbf{x} + \Delta b).$$

# Perceptron learning rule

- Suppose we see a misclassified example  $(\mathbf{x}, y)$  where  $y(\mathbf{w}^\top \mathbf{x} + b) \leq 0$ .
- We want to update  $(\mathbf{w}, b)$  so that the signed margin increases:  $y(\mathbf{w}\mathbf{x}+b)$  should go up.
- $\mathbf{w}' = \mathbf{w} + \Delta\mathbf{w}, b' = b + \Delta b$ .
- New signed margin

$$y(\mathbf{w}'^\top \mathbf{x} + b') = y((\mathbf{w} + \Delta\mathbf{w})^\top \mathbf{x} + (b + \Delta b)) = y(\mathbf{w}^\top \mathbf{x} + b) + y(\Delta\mathbf{w}^\top \mathbf{x} + \Delta b).$$

- simple choice that guarantees an increase is to pick  $\Delta\mathbf{w}$  **parallel to**  $y\mathbf{x}$  and  $\Delta b$  parallel to  $y$      **$\Delta\mathbf{w} = \eta y \mathbf{x}$ ,  $\Delta b = \eta y$**

$$y(\Delta\mathbf{w}^\top \mathbf{x} + \Delta b) = y(\eta y \mathbf{x}^\top \mathbf{x} + \eta y) = \eta(\|\mathbf{x}\|^2 + 1) > 0.$$

$$\boxed{\mathbf{w} \leftarrow \mathbf{w} + \eta y \mathbf{x}}, \quad \boxed{b \leftarrow b + \eta y}$$

# Perceptron Learning rule

- Gradient decent perspective
- Consider a loss function

$$\ell(\mathbf{w}, b; \mathbf{x}, y) = \max(0, -y(\mathbf{w}^\top \mathbf{x} + b)).$$

$$L_{\text{hinge}}(y, f(x)) = \max(0, 1 - y(w \cdot x + b))$$

- When  $y(\mathbf{w}^\top \mathbf{x} + b) \leq 0$ , the loss is linear  $\ell = -y(\mathbf{w}^\top \mathbf{x} + b)$ .

$$\nabla_{\mathbf{w}} \ell = -y\mathbf{x}, \quad \frac{\partial \ell}{\partial b} = -y.$$

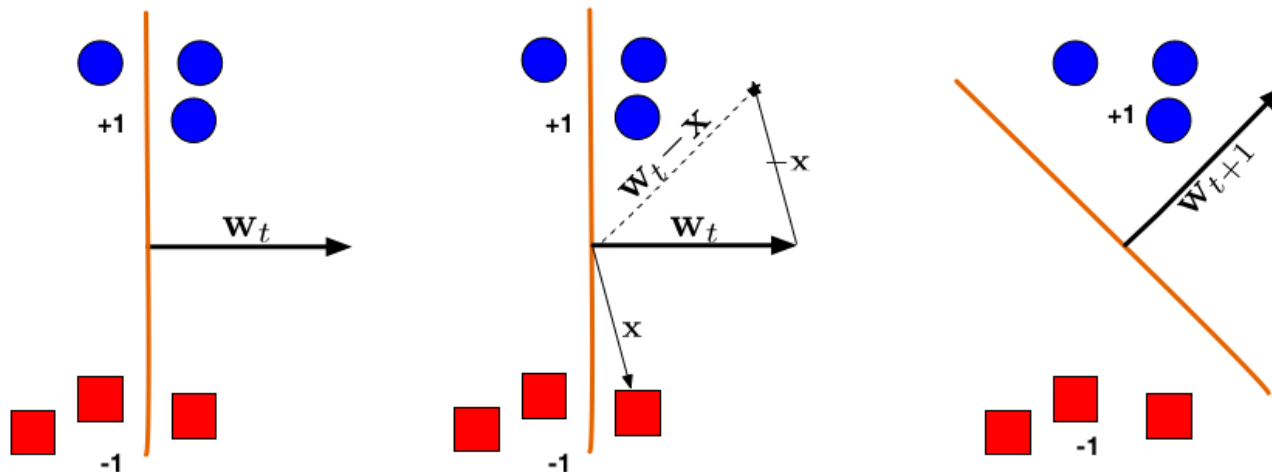
- Gradient descent step

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \ell = \mathbf{w} - \eta(-y\mathbf{x}) = \mathbf{w} + \eta y\mathbf{x},$$

$$b \leftarrow b - \eta \frac{\partial \ell}{\partial b} = b - \eta(-y) = b + \eta y.$$

# Perceptron Learning

- Geometrical interpretation
    - Positive example misclassified :  $w \leftarrow w + \eta x \Rightarrow w$  is rotated **toward**  $x$ , increases  $w^\top x$ .
    - Negative example misclassified :  $w \leftarrow w - \eta x \Rightarrow w$  is rotated **away from**  $x$ , decreases  $w^\top x$ .
    - Bias update as a parallel shift in hyperplane
- Colab [demo](#) reference



# Perceptrons

- Converges for linearly separable data
- Convergence result: For linearly separable dataset, the algorithm converges after finite iterations.
- Generalized to non-binary (real) input
- Provided a learning algorithm
- What if the data is not linearly separable?